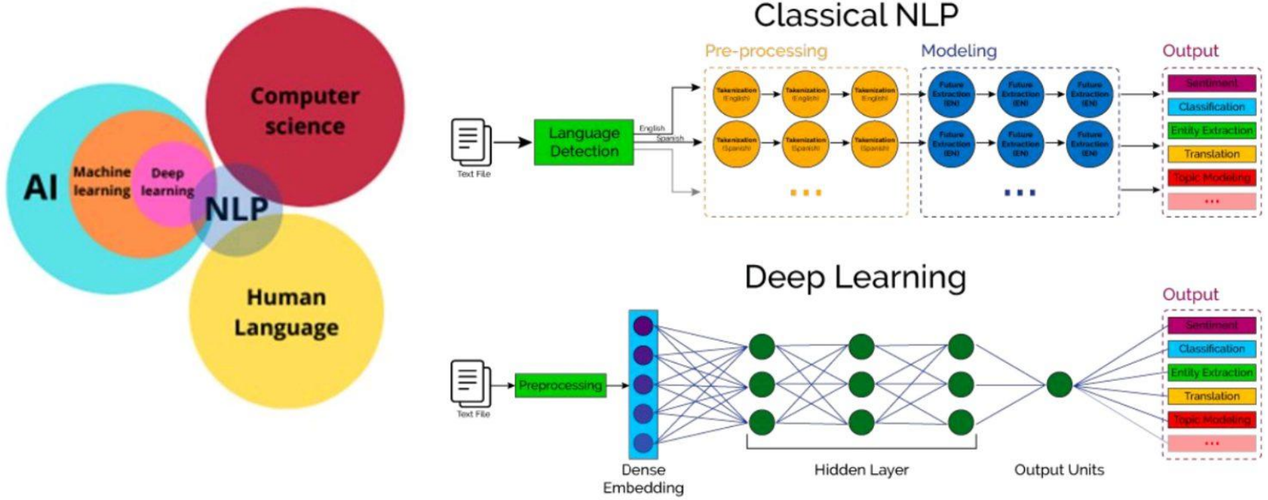
Natural Language Processing



Natural Language Processing

The Bag of Words Representation

I love this movie! It's sweet, but with satirical humor. The dialogue is great and the adventure scenes are fun... It manages to be whimsical and romantic while laughing at the conventions of the fairy tale genre. I would recommend it to just about anyone. I've seen it several times, and I'm always happy to see it again whenever I have a friend who hasn't seen it yet!



it	6
I	5
the	4
to	3
and	3
seen	2
yet	1
would	1
whimsical	1
times	1
sweet	1
satirical	1
adventure	1
genre	1
fairy	1
humor	1
have	1
great	1
…	…

Bag of words

```
corpus = [
    "This is the best book I have ever read",
    "Reading books is the best way to gain knowledge",
    "Which book you prefer. This one or the another one?",
    "Is this the book you love most?"
]
```

Make vocabulary

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21
another	best	book	books	ever	gain	have	is	knowledge	love	most	one	or	prefer	read	reading	the	this	to	way	which	you

Score assignment

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21
0	1	1	0	1	0	1	1	0	0	0	0	0	0	1	0	1	1	0	0	0	0
0	1	0	1	0	1	0	1	1	0	0	0	0	0	0	1	1	0	1	1	0	0
1	0	1	0	0	0	0	0	0	0	0	2	1	1	0	0	1	1	0	0	1	1
0	0	1	0	0	0	0	1	0	1	1	0	0	0	0	0	1	1	0	0	0	1

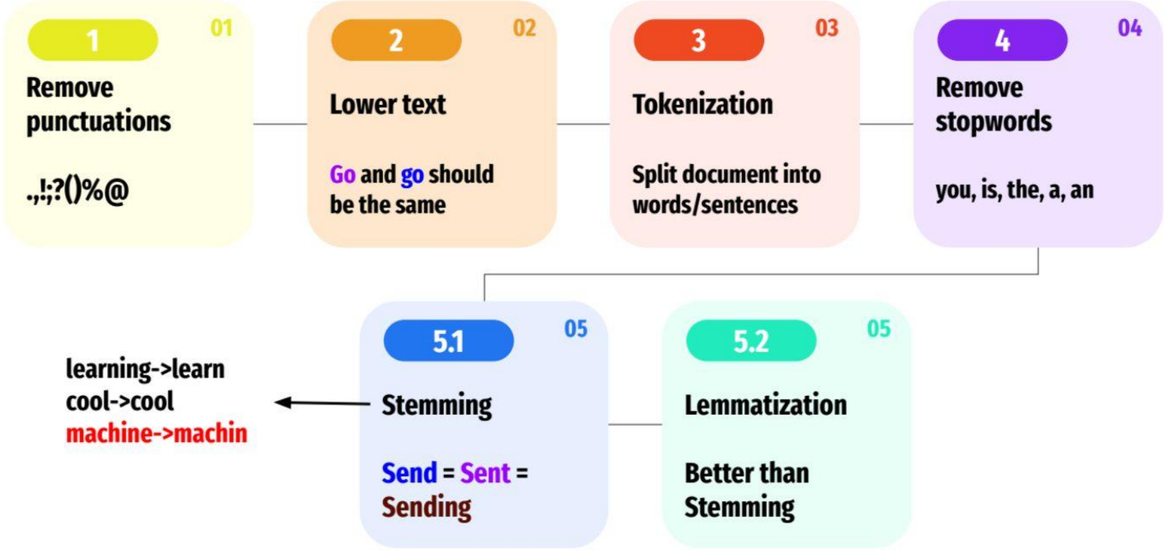
CountVectorizer

```python
from sklearn.feature_extraction.text import CountVectorizer

corpus = [
    "This is the best book I have ever read",
    "Reading books is the best way to gain knowledge",
    "Which book you prefer. This one or the another one?",
    "Is this the book you love most?"
]
vectorizer = CountVectorizer()
data = vectorizer.fit_transform(corpus)
vocabulary = vectorizer.get_feature_names_out()
print("Vocabulary:")
print(vocabulary)
result = data.toarray()
print("Bag of words:")
print(result)
```

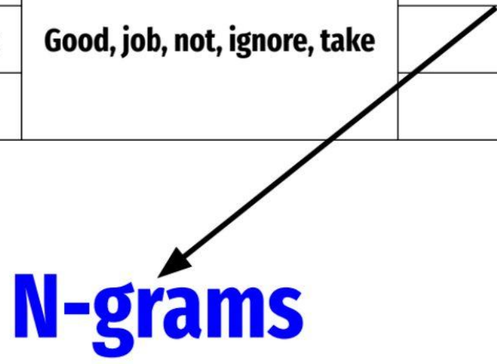
```
example
C:\Users\Viet\anaconda3\envs\basic\python.exe C:\Users\Viet\PycharmProjects\basic\example.py
Vocabulary:
['another' 'best' 'book' 'books' 'ever' 'gain' 'have' 'is' 'knowledge'
 'love' 'most' 'one' 'or' 'prefer' 'read' 'reading' 'the' 'this' 'to'
 'way' 'which' 'you']
Bag of words:
[[0 1 1 0 1 0 1 1 0 0 0 0 0 1 0 1 1 0 0 0]
 [0 1 0 1 0 1 0 1 1 0 0 0 0 0 1 1 0 1 1 0]
 [1 0 1 0 0 0 0 0 0 0 2 1 1 0 0 1 1 0 0 1]
 [0 0 1 0 0 0 0 1 0 1 1 0 0 0 0 0 1 1 0 0 1]]
```

Text preprocessing



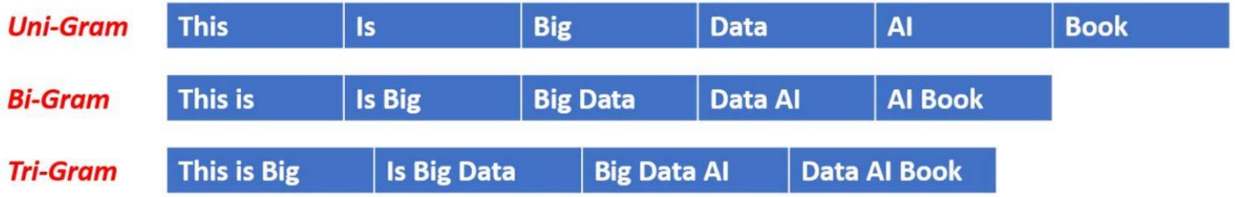
Bag of words

Document	Vocabulary (no stopwords)	Bag of words' vector
This is a good job. I will not ignore it	Good, job, not, ignore, take	1,1,1,1,0
This is not a good job. I will ignore it		1,1,1,1,0
This is a good job. I will take it		1,1,0,0,1



N-grams

This is Big Data AI Book



CountVectorizer

```python
from sklearn.feature_extraction.text import CountVectorizer

corpus = [
    "This is the best book I have ever read",
    "Reading books is the best way to gain knowledge",
    "Which book you prefer. This one or the another one?",
    "Is this the book you love most?"
]
vectorizer = CountVectorizer(ngram_range=(1, 2), stop_words="english")
```

```
Vocabulary:
['another' 'another one' 'best' 'best book' 'best way' 'book' 'book have'
 'book you' 'books' 'books is' 'ever' 'ever read' 'gain' 'gain knowledge'
 'have' 'have ever' 'is' 'is the' 'is this' 'knowledge' 'love' 'love most'
 'most' 'one' 'one or' 'or' 'or the' 'prefer' 'prefer this' 'read'
 'reading' 'reading books' 'the' 'the another' 'the best' 'the book'
 'this' 'this is' 'this one' 'this the' 'to' 'to gain' 'way' 'way to'
 'which' 'which book' 'you' 'you love' 'you prefer']
Bag of words:
[[0 0 1 1 0 1 1 0 0 0 1 1 0 0 1 1 1 1 0 0 0 0 0 0 0 0 0 0 1 0 0 1 0 1 0
  1 1 0 0 0 0 0 0 0 0 0 0 0]
 [0 0 1 0 1 0 0 0 1 1 0 0 1 1 0 0 1 1 0 1 0 0 0 0 0 0 0 0 0 1 1 1 0 1 0
  0 0 0 0 1 1 1 1 0 0 0 0 0]
 [1 1 0 0 0 1 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 2 1 1 1 1 1 0 0 0 1 1 0 0
  1 0 1 0 0 0 0 0 1 1 1 0 1]
 [0 0 0 0 0 1 0 1 0 0 0 0 0 0 0 1 0 1 0 1 1 1 0 0 0 0 0 0 0 0 0 1 0 0 1
  1 0 0 1 0 0 0 0 0 0 1 1 0]]
```

Term Frequency-Inverse Document Frequency

$$TF = \frac{Number\ of\ times\ a\ word\ "X"\ appears\ in\ a\ Document}{Number\ of\ words\ present\ in\ a\ Document}$$

$$IDF = log\left(\frac{Number\ of\ Documents\ present\ in\ a\ Corpus}{Number\ of\ Documents\ where\ word\ "X"\ has\ appeared}\right)$$

$$TF\ IDF\ =\ TF * IDF$$

Term Frequency (TF)

Tần số xuất hiện của 1 word trong 1 document

Inverse Document Frequency (IDF)

Mức độ phổ biến/hiếm của 1 word trong toàn bộ các documents